

Hackhub Game Handbook

 Handbook

 Contents

 Search the handbook...

 Search

Contents

Guides & Reference.



Accounts
1 articles



Digging
2 articles



Metasploit
6 articles



Scripting
14 articles



Suspicion
4 articles



Terminal Commands
32 articles



Python Modules
9 articles



Online World
21 articles

Extracted from the in-game Handbook · 2026-09-19

Contents

Accounts 1 articles

- Account Recovery

Digging 2 articles

- Finding User Passwords
- Hijack Firewall

Metasploit 6 articles

- Intro
- How To Hack
- Example Hacking
- Rootgrab
- Payloads
- Reverse TCP

Scripting 14 articles

- Create Your Own Program
- Convert Code to Command
- Modules & Imports
- System
- Shell
- Packages
- Debug
- Thread
- File System
- Networking
- Networking.Wifi
- Crypto
- HackDB
- NTLM

Suspicion 4 articles

- What is Suspicion?
- How does my level of Suspicion increase?
- How to reduce?
- High level of Suspicion

Terminal Commands 32 articles

- apt-get install
- bettercap
- cat
- cd
- cp
- dig
- dirhunter
- download
- ftp
- geoip
- git
- hydra
- john
- ls
- lynx
- mkdir
- mv
- mxlookup
- nmap
- node
- nslookup
- nuclei
- openssl
- ping
- pip
- python3
- rm
- ssh
- subfinder
- sudo
- weechat
- whois

Python Modules 9 articles

- Evil-Rm
- Fern
- JWT Decoder
- Kimai
- Net Tree
- Pret
- pyUserEnum
- Sqlmap
- WigleNet

Online World 21 articles

- The Online World
- Finding Targets
- Targets & Tiers
- Tier 1: Direct Targets
- Tier 2: VPN-Gated Targets
- Tier 3: Vaults
- Breaking In
- VPN Certificates
- Keys & Decoders
- Vaults & Pivoting
- Loot Packages
- Appraising Loot
- Encrypted Loot
- Cashing Out
- The GreyMarket
- Heat & Hot Loot
- Getting Traced
- Proxies & Rigs
- Your Own Infrastructure
- Working From a Rented Server
- Operator Commands

How to recover your account?

If you have created an account, the system automatically keeps a log of it. To see these logs, review the `/logs/accounts.log` file.

```
/logs/accounts.log
```

How to Find User Password in Remote Connection?

When you access a system, you may need users' passwords. Passwords are stored in an encrypted form in the operating system.

These passwords are stored in this file:

```
/etc/passwd
```

If you want, you can decrypt this file completely or the hashes in it one by one.

To do this, a tool is required.

```
apt-get install john
```

then

```
john <hash>
```

Identifying the Firewall

Before we begin the credential extraction procedure, we must determine the firewall's IP address.

Let's assume the target firewall IP is **44.55.66.77**.

Required Tools

To perform this operation, you will need the following:

- **Wireshark** — Used for monitoring network packets
- **Kimai Attack Script** — Downloadable from *hackdb.net*
- **JWT Decoder** — Also available at *hackdb.net*

These three tools work together to capture and decode authentication tokens.

Preparing Wireshark

Open **Wireshark** and configure it for maximum visibility.

In the **Source** and **Destination** filters, enter:

```
*
```

This allows Wireshark to capture all incoming and outgoing data without restriction.

After setting the filter, press **Start** to begin live packet capturing.

You should now see a constant flow of traffic in the packet list.

Generating Traffic With Kimai

Next, we must generate activity toward the target firewall to force it to produce session-related data. From your terminal, launch the Kimai script targeting the firewall:

```
python3 ./kimai.py [ip address]
```

Example:

```
python3 ./kimai.py 44.55.66.77
```

Each execution *may* or may not result in a captured cookie.

Because of this, you must run the Kimai script repeatedly until a usable token appears in Wireshark. While Kimai is running, Wireshark should display a continuous stream of packets.

Finding the Cookie in Wireshark

Monitor the **Info** column in Wireshark.

When a cookie or token appears, it will usually be displayed as a short encoded string or session reference.

Continue running Kimai and monitoring the packets until a cookie-like entry becomes visible.

Once it appears, the data has been successfully captured.

Click on the packet to view its detailed contents, then copy the token.

Decoding the Token

Now that you have the captured token, use the JWT Decoder tool.

Run the decoder script as follows:

```
python3 ./jwt_decoder.py [token]
```

Example:

```
python3 ./jwt_decoder.py eyJhbGciOiJIUzI1NiIsInR5cCI6...
```

The decoder will process the token and reveal its contents.

Among the decoded fields, you will find the firewall's login credentials.

Conclusion

At this stage:

1. The firewall IP was identified 2. Wireshark was configured to capture all traffic 3. Kimai generated forced traffic to obtain a token 4. A cookie was found inside Wireshark capture 5. The token was

decoded using the JWT Decoder 6. Firewall login credentials were extracted successfully

Introduction to Metasploit

Metasploit is a penetration testing tool used to detect and test security vulnerabilities.

It is used by ethical hackers and cybersecurity professionals to identify weaknesses in systems and improve defenses.

However, it can also be used by malicious actors for attacks, so it should be used carefully and within legal boundaries.

Installation

```
apt-get install metasploit
```

Usage

```
msfconsole
```

How to Hack with Metasploit?

Metasploit is divided into exploits. Although the usage of each exploit is different, it works in more or less the same way.

We cannot use Metasploit alone, we need to use some tools along with it.

For example, we need to use **nmap** to learn which services are running on the system we want to infiltrate, which ports are active.

According to the results we found, if a port is open and we know the running version, we can usually infiltrate this system.

Let's get started.

Scanning the Target

For example, let's say our target's IP address is **11.22.33.44**.

We should start our hack by scanning the target first.

```
nmap 11.22.33.44 -sV
```

The result we got:

PORT	STATE	SERVICE	VERSION	DESTINATION	----	----	----	----	----		21	OPEN		
ftp	vsftpd	2.3.5	192.168.1.2		22	CLOSE	ssh	openssh	1.8.6	192.168.1.2		80	CLOSE	http
nginx	7.23.4	192.168.1.2												

From here we understand that port 21 is open and the service running in the background is "**vsftpd version 2.3.5**".

Now we can start using metasploit.

```
msfconsole
```

Searching Exploit

We know that our target has an open port and a running service. Next, we need to find an exploit that fits it.

```
search [running service name]
```

"vsftpd 2.3.5" was running on our target port.

We need to look for a suitable exploit for vsftpd.

```
search vsftpd
```

```
| # | Name | Disclosure Date | Rank | Description | | ---- | ---- | ---- | ---- | ---- | | 0 |
auxiliary/dos/ftp/vsftpd | 2020-02-03 | normal | VSFTPD Denial of Service |
```

We found a usable exploit. We need to test it.

In order to use the exploit we found after searching, we need to use the **"use"** command.

```
use [exploit name or index]
```

For this example

```
use 0
```

Or

```
use auxiliary/dos/ftp/vsftpd
```

Making Configurations

We found our exploit and used it with the **"use"** command.

Now we need to configure the module we are using.

```
show options
```

After using this command we displayed the current settings of the module.

```
| Name | Current Settings | Required | Description | | ---- | ---- | ---- | ---- | | RHOST | | yes | The target host | | RPORT | 21 | yes | The target host | | Version | 1.0.0 | yes | The target host |
```

We must fill in the fields with Required = **"yes"**

We fill these fields with the **"set"** command.

```
set RHOST 11.22.33.44
```

```
set Version 2.3.5
```

Note: Some fields may be filled by default, you should replace them with information about your target.

Once you have filled in all the required fields you can start the attack.

Attacking

Now that we have made all the settings, we can start the attack.

Usually this command is **"exploit"**. However, sometimes there may be other commands. You can learn by typing "help".

```
exploit
```

If everything is correct you will have infiltrated the system and metasploit will give you a **meterpreter** session.

Rootgrab

This metasploit tool get root password hash from passwd file.

Usage (In meterpreter) `rootgrab /path/to/passwd`

Example `rootgrab /etc/passwd`

Hint Usually the passwd file is located under /etc.

Metasploit Payloads

You can add payloads to some Metasploit exploits. These payloads are code scripts that will run on the target system.

The most well-known is `bearos/meterpreter/reverse_tcp`

Usage `set payload <payload name>`

Example `set payload bearos/meterpreter/reverse_tcp`

Metasploit Reverse TCP

In some cases, most of the paths to the target system are closed, and we can't access them with our attacks. However, the target can open a way for us to enter, called Reverse TCP.

The most common way to deceive someone is through files. Seemingly harmless Word documents can carry malware that doesn't appear to be hidden.

The exploit we will use here is `exploit/multi/fileformat/office_word_macro`

Usage

- `search office_word_macro`

- `use exploit/multi/fileformat/office_word_macro`
- `set payload bearos/meterpreter/reverse_tcp`
- `show options`
- `set LHOST 192.168.1.2`
- `set LPORT 4444`
- `run` (to create a malware word document)
- `handler` (to start the listener)
- Send file to target

When the connection comes

- With *Ctrl+C* exit handler.
- `show sessions`
- `session <session index>`

Important
Here, we've set the payload's LPORT (i.e., local port) to 4444. You can set this to any port you like. However, remember, you must open this port from the modem interface of the network you're connected to. Otherwise, the payload won't work.

How do I create my own program?

You can automate hacking or other things by creating your own custom programs.

You can do this with the **Code++** application.

Hackhub supports **TypeScript** language for scripting.

You can use the default TypeScript syntax. Additionally, Hackhub has functions and libraries defined for managing the game.

Code++ Editor Features

- **Tabbed editing** – Open and work on multiple files at once. Drag tabs to reorder them.
- **Themes** – Choose from 20 color themes (Dracula, Tokyo Night, Catppuccin, etc.) in Settings.
- **Fonts** – Pick your preferred font family and size in Settings.
- **IntelliSense** – Full autocomplete and type checking for all built-in APIs.
- **Import support** – Split code across files with `import` / `export` (see "Modules & Imports").

I wrote the code, But how do I run it?

After writing a program, you need the "node.js" package to run it.

```
apt-get install node
```

Then run the typescript file you saved.

```
node /path/to/yourprogram.ts
```

You can also press the **Play** button in Code++ to run the active file directly.

Converting code to command

After writing a program, we run it with the "node" command. So how do we turn this program into a terminal command?

If you put the code file you created into the "lib" folder, that file will now run as a special command.

Example:

```
/lib/myprogram.ts
```

In terminal:

```
myprogram
```

Another example:

```
/lib/custom-tool.ts
```

custom-tool

Modules & Imports

You can split your code across multiple files and import/export between them using standard TypeScript syntax.

Exporting

Use `export` to make functions, classes, or variables available to other scripts.

```
// utils.ts
export function greet(name: string) {
    println("Hello, " + name + "!");
}

export const VERSION = "1.0";

export async function doWork() {
    println("Working...");
    await sleep(1000);
    println("Done.");
}
```

Importing

Use `import` to bring in exports from another file. Paths are relative to the current file.

```
// main.ts
import { greet, VERSION, doWork } from "./utils";

greet("World");
println("Version: " + VERSION);
await doWork();
```

How It Works

- Paths are relative to the importing file: `./utils` is a sibling, `./lib/http` sits in a subfolder and `../utils/buzz` is one folder up.
- The `.ts` extension is optional – `./utils` and `./utils.ts` both work.

- A folder is a module of its own when it holds an `index.ts`, so `"../utils"` reads

```
../utils/index.ts
```

- All built-in functions (`println`, `sleep`, `prompt`, `Shell`, `Networking`, etc.) work inside imported modules.
- Imports are resolved recursively – imported files can import other files.

Folders With An Index File

An `index.ts` lets a folder decide what it hands out, and the files importing it stop caring how it is arranged inside.

```
// utils/index.ts
export { fizz } from "./fizz";
export { buzz } from "./buzz";
```

```
// scripts/main.ts
import { fizz, buzz } from "../utils";

println(fizz() + buzz());
```

Multi-File Project Example

```
// scanner.ts
export async function scanTarget(ip: string) {
  const subnet = await Networking.GetSubnet(ip);
  if (!subnet) throw "No subnet found!";
  const ports = await subnet.GetPorts();
  return ports;
}
```

```
// main.ts
import { scanTarget } from "./scanner";

const ip = await prompt("Target IP: ");
const ports = await scanTarget(ip);
println("Open ports: " + ports.join(", "));
```

Type Support

The Code++ editor provides full IntelliSense for imports, wherever the file lives. When you type

```
import { } from "./file"
```

, the editor autocompletes the symbols that file exports, and the same

works for a file in another folder or a folder with an `index.ts`.

Lib – System

Basic I/O functions for your scripts.

Printing Output

```
println("Hello World!");  
println(42);
```

Colored Output

```
println({ text: "Success!", color: "green" });  
println({ text: "Error!", color: "red", backgroundColor: "#333" });
```

Inline Colored Text

```
println([  
  { text: "Status: ", color: "white" },  
  { text: "OK", color: "green" }  
]);
```

Print Table

Display data in a formatted ASCII table.

```
printTable([  
  { Name: "SSH", Port: 22, Status: "open" },  
  { Name: "HTTP", Port: 80, Status: "closed" }  
]);
```

New Line

```
newLine();
```

User Input

```
const ip = await prompt("IP Address: ");  
println(`You entered: ${ip}`);
```

Password Input (hidden)

```
const pass = await prompt({ label: "Password: ", password: true });
```

Lib – Shell

Manage the terminal from your scripts.

Terminal Lock

Lock terminal input during long operations.

```
await Shell.lock();    // Locks input
await Shell.unlock();  // Unlocks input
await Shell.isLocked(); // -> boolean
```

Clear Terminal

```
await Shell.clear();
```

Arguments

Get command-line arguments passed to your script.

Example: `myprogram -ip 192.168.1.1 -port 21`

```
const args = Shell.GetArgs();
// -> ["-ip", "192.168.1.1", "-port", "21"]
```

Execute Commands

Run terminal commands from your script.

```
await Shell.Process.exec("nmap 192.168.1.1 -sV");
```

With working directory options:

```
// Run in a specific subdirectory
await Shell.Process.exec("cat passwords.txt", {
  cwd: "documents"
});

// Run with absolute path
await Shell.Process.exec("ls", {
  cwd: "/home/user/desktop",
  absolute: true
});
```

Reading Output

`exec()` returns the command's output as text, so your script can use it.

```
const output = await Shell.Process.exec("nslookup freshfoods.com");
const ip = output.match(/\d+\.\d+\.\d+\.\d+/)?.[0];
println(`IP Address: ${ip}`);
```

Use `silent` when you only want the value and not the output on screen.

```
const output = await Shell.Process.exec("nslookup freshfoods.com", {
  silent: true
});
```

Lib – Packages

To check if a package is installed

```
checkLib(packageName: string) -> Promise<boolean>
```

Example

```
await checkLib("lynx")
```

Returns true if package installed, and false for not installed.

Installing Package

```
installLib(packageName: string) -> Promise<boolean>
```

Example

```
const isInstalled = await installLib("lynx");  
println(isInstalled); // Returns true or false.
```

Lib – Debug

You can use the game's developer console to log your output. Use the "F1" key to do this.

```
Debug.Log("Hello World!");
```

```
Debug.Error("This is error!");
```

Lib – Thread

Used for thread operations.

To wait

```
sleep(timeout: number (ms)) -> Promise<void>
```

```
println("Loading...");  
await sleep(2500); // Waits 2.5 seconds  
println("Done.");
```

Lib – File System

Manage files and directories on the computer.

Most methods accept an optional **options** parameter:

- **absolute** – resolve path from root instead of current directory
- **recursive** – create intermediate directories automatically (Mkdir, WriteFile)

Current Working Directory

```
const info = await FileSystem.cwd();
println(info.name);           // "desktop"
println(info.currentPath);    // "desktop"
println(info.absolutePath);   // "/home/user/desktop"
```

Navigate Directories

```
await FileSystem.SetPath("documents");
await FileSystem.SetPath("/home/user", { absolute: true });
```

List Files

```
const files = await FileSystem.ReadDir(".");
if (!files) throw "Cannot access directory!";
for (const file of files) {
    println(`${file.name} - ${file.isFolder ? "folder" : file.extension}`);
}
```

Read File

```
const content = await FileSystem.ReadFile("notes.txt");
println(content);
```

Write File

Creates a new file or overwrites an existing one.

```
await FileSystem.WriteFile("output.txt", "Hello World!");

// Auto-create parent directories
await FileSystem.WriteFile("deep/path/file.txt", data, {
    recursive: true
});
```

Create Directory

```
await FileSystem.Mkdir("my-folder");
await FileSystem.Mkdir("path/to/folder", { recursive: true });
```

Delete File or Folder

```
await FileSystem.Remove("old-file.txt");
```

Lib – Networking

Network operations and subnet analysis.

Validate IP Address

```
await Networking.IsIp("192.168.1.1"); // true
await Networking.IsIp("abc");         // false
```

Get Subnet

```
const subnet = await Networking.GetSubnet("11.22.33.44");
if (!subnet) throw "Subnet not found!";
println(subnet.ip);    // "11.22.33.44"
println(subnet.lanIp); // "192.168.1.2"
```

Subnet Methods

```
const subnet = await Networking.GetSubnet(ip);

// Get the router of this subnet
const router = await subnet.GetRouter();

// List open port numbers
const ports = await subnet.GetPorts(); // -> number[]

// Check if a specific port is open
const isOpen = await subnet.PingPort(22); // -> boolean

// Get detailed port information
const portData = await subnet.GetPortData(22);
if (portData) {
    println(portData.service); // "ssh"
    println(portData.version); // "OpenSSH 8.2"
    println(portData.external); // 22
    println(portData.internal); // 22
    println(portData.target);   // "192.168.1.2"
}
```

Lib – Networking.Wifi

Wireless network scanning and attack tools (similar to Bettercap).

List Interfaces

```
const interfaces = await Networking.Wifi.GetInterfaces();
for (const iface of interfaces) {
    println(`${iface.name} - monitor: ${iface.monitor}`);
}
// wlan0 - monitor: true
// wlan1 - monitor: false
```

Scan Networks

Requires a monitor-mode capable interface (e.g. `wlan0`).

```
const networks = await Networking.Wifi.Scan("wlan0");
for (const ap of networks) {
  println(`${ap.ssid} | ${ap.bssid} | Ch:${ap.channel} | Signal:${ap.signal}`);
}
```

Deauthentication Attack

Send deauth frames to disconnect clients from an access point.

```
await Networking.Wifi.Deauth("wlan0", targetBssid);

// With custom packet count
await Networking.Wifi.Deauth("wlan0", targetBssid, { packets: 10 });
```

Capture WPA Handshake

Captures the WPA handshake and saves it as a `.pcap` file in the current directory.

```
const pcapFile = await Networking.Wifi.CaptureHandshake("wlan0", bssid);
println(`Saved: ${pcapFile}`); // "networkname.pcap"
```

Full Example – Wi-Fi Cracker

```
const ifaces = await Networking.Wifi.GetInterfaces();
const mon = ifaces.find(i => i.monitor);

const networks = await Networking.Wifi.Scan(mon.name);
const target = networks[0];

await Networking.Wifi.Deauth(mon.name, target.bssid);
const pcap = await Networking.Wifi.CaptureHandshake(mon.name, target.bssid);

const password = await Crypto.Hashcat.Decrypt(pcap);
println(`Password: ${password}`);

await Networking.Wifi.Connect(target.bssid, password);
```

Connect

Joins an access point by SSID or BSSID. Throws when the target cannot be found, when several networks share the same SSID, or when the passphrase is rejected.

```
await Networking.Wifi.Connect("HOME_5Ghz", "letmein123");

// Leave the passphrase out and you are asked for it on the terminal (masked input)
await Networking.Wifi.Connect(target.bssid);
```

Current Connection

```
const ap = await Networking.Wifi.GetCurrent();
println(ap ? `Connected: ${ap.ssid}` : "Offline");
```

Disconnect

Returns `false` when there was no connection to drop.

```
await Networking.Wifi.Disconnect();
```

Lib – Crypto

Cryptographic utilities and password cracking.

Hashcat – Crack Wi-Fi Passwords

Decrypt captured `.pcap` files to reveal Wi-Fi passwords.

```
const password = await Crypto.Hashcat.Decrypt("capture.pcap");
println(`Cracked: ${password}`);
```

With path options:

```
// Resolve from a specific directory
await Crypto.Hashcat.Decrypt("capture.pcap", { cwd: "downloads" });

// Resolve from root
await Crypto.Hashcat.Decrypt("capture.pcap", { absolute: true });
```

Hash Utilities

```
// Generate MD5 hash
const hash = Crypto.Hash.md5("hello world");

// Encrypt a password (stores mapping for later decryption)
const encrypted = await Crypto.Hash.encrypt("mypassword");

// Decrypt a previously encrypted hash
const original = await Crypto.Hash.decrypt(encrypted);
println(original); // "mypassword"
```

Lib – HackDB

Access the exploit database programmatically.

List All Exploits

```
const exploits = await HackDB.ListExploits();
for (const exploit of exploits) {
```

```
println(`${exploit.title} - ${exploit.service} ${exploit.version}`);
}
```

Search Exploits

```
const results = await HackDB.SearchExploits("ftp");
printTable(results.map(e => ({
  Title: e.title,
  Service: e.service,
  Version: e.version
})));
```

Download Exploit

```
// Download to ~/downloads (default)
await HackDB.DownloadExploit("sqlmap");

// Download to a specific directory
await HackDB.DownloadExploit("wordlist", "tools");

// Download with absolute path
await HackDB.DownloadExploit("kimai", "/home/user/desktop", {
  absolute: true
});
```

Lib – NTLM

NTLM authentication attack library.

Check NTLM Service

Check if a target has NTLM authentication enabled.

```
const hasNtlm = await NTLM.Check("11.22.33.44");
if (!hasNtlm) throw "No NTLM service found.";
```

Connect & Dump Hashes

```
const connection = await NTLM.Connect("11.22.33.44");
const instance = await connection.GetInstance("2"); // NTLM version
await instance.Dump("admin"); // Dumps the user's NTLM hash
```

Full Example

```
const ip = "11.22.33.44";

if (await NTLM.Check(ip)) {
  println("NTLM service found, connecting...");
  const conn = await NTLM.Connect(ip);
  const ntlm = await conn.GetInstance("2");
  await ntlm.Dump("admin");
}
```

```
println("NTLM hash dumped successfully.");  
}
```

What is Suspicion Level?

Suspicion level, as the name suggests, indicates your level of suspicion in the game.

Be careful what you do when hacking or baiting someone!

How does my level of Suspicion increase?

Your level of suspicion in the game usually increases when you make an incomplete or incorrect action during the hacking phase or while sending phishing emails.

Hacking

- When you access a system, you need to find the access logs.
- Afterwards, you should delete the log stating that you have obtained a shell and not touch the other logs, otherwise the missing logs will be noticed by the system administrators.

Phishing Emails

- When sending a phishing email, make sure you enter the data correctly or you will attract suspicion!

How to reduce Suspicion?

- You can change your connected Wi-Fi network, this will change the IP and DNS address, which will reduce your suspicion level.

What happens at high levels of suspicion?

- If your suspicion level is 50% or higher, there is a possibility that you can be hacked at any time. These hacks include mini games, if you succeed in these mini games, you will avoid the hack, if you fail, you will be penalized such as losing your money.
- If the suspicion level reaches 100%, you will be seized by federal authorities and lose all your money.

Command: **apt-get install**

Installs terminal command package.

Usage

```
apt-get install [package name]
```

Command: **bettercap**

Bettercap is a powerful, modular, and easily extensible framework designed for network attacks and monitoring.

Installation

```
apt-get install bettercap
```

Start

```
bettercap
```

Start Probing

```
net.probe on
```

Show network card list

```
net.show
```

Set wi-fi recon to interface which monitoring mode is enabled

```
wifi.recon [interface]
```

Then show wi-fi list

```
wifi.show
```

Set target wi-fi

```
set wifi.ap [BSSID]
```

Send deauthentication packets to target

```
wifi.deauth
```

Exit bettercap

```
exit
```

Install hashcat for decrypt hash

```
apt-get install hashcat
```

Crack password

```
hashcat [pcap file]
```

Command: cat

Reads file content.

Usage

```
cat [file name]
```

Command: cd

The cd command used to change the current working directory.

Usage

```
cd [directory name]
```

To go back or upper directory:

```
cd ..
```

Command: cp

Copy files and directories.

Usage

```
cp <source> <destination>
```

Command: dig

Dig is a command-line tool for gathering information about IP address.

```
dig <ip address>
```

Command: dirhunter

Dirhunter is a web directory discovery tool. It scans a target website and reveals its publicly accessible paths and pages.

Useful for reconnaissance – finding hidden admin panels, API endpoints, or forgotten pages on a target domain.

Install: `apt-get install dirhunter`

Usage: `dirhunter <domain>`

Example: `dirhunter example.com`

The tool will scan the target and list all discovered directories and pages.

Command: download

Downloads a file from the remote server to your local machine.

Usage

`download <file path>`

Command: ftp

It allows us to connect and manage the remote ftp server.

Usage

`ftp -h [host] -u [user] -p [password]`

Command: geoiip

Geoiip is a well-known tool that helps you find the location of a given IP address.

`geoiip <ip address>`

Example `geoiip 46.20.15.34`

Command: git

Git is the platform where players store and share the programs they write.

Initialize Repository For example we're on folder that have scripts. `git init -n <repo name>`

Commit Add commit to repository. `git commit -m <message>`

Push Push commit to repository. `git push`

Pull Pull latest updates on repository. `git pull`

Clone Clone repository. `git clone <repository slug> <path to clone>`

Example `git clone example-hack-tool ./home/user/desktop/repo-folder`

Command: hydra

Hydra is a parallelized login cracker that supports numerous protocols.

- The `-u` flag is optional. If omitted, Hydra automatically uses **root** as the default username.
- The `-T` flag specifies the target in the `_IP:Port_` format.

Example: `hydra -T 192.168.1.1:22 -P ./wordlist.lst`

- The `-P` flag is used to provide the path to your wordlist file. Make sure to specify the correct absolute or relative path.

Example: if you're in the root directory and the file is inside **downloads**:

```
-P /downloads/wordlist.lst
```

Usage (default username: root): `hydra -T [ip:port] -P [wordlist]`

Usage (with username): `hydra -T [ip:port] -l [username] -P [wordlist]`

Command: john

john, meaning "John The Ripper", is used to crack the password of encrypted texts.

You must install the package first.

```
apt-get install john
```

john has 2 different uses.

- File password cracking

```
john /etc/passwd
```

- Deciphering the hash string

```
john <hash>
```

Example

```
john 31435008693ce6976f45dedc5532e2c1
```

Command: ls

The ls command is used to list files. "ls" on its own lists all files in the current directory.

Usage

```
ls
```

Command: lynx

It performs a deep search on the Internet.

Usage

```
lynx [search]
```

Creates a new directory in the current working path.

```
mkdir <directory name>
```

Command: mv

Move or rename files and directories.

Usage

```
mv <source> <destination>
```

Command: mxlookup

Finds mail server of domain.

```
mxlookup <domain>
```

Example

```
mxlookup example.com
```

Command: nmap

Nmap is an open source tool used for network scanning and vulnerability detection.

You can perform a more detailed scan using the `-sV` flag and learn about the services running in the background and their version information.

Default usage

```
nmap [ip address]
```

For detailed usage

```
nmap [ip address] -sV
```

For specific port

```
nmap [ip address] -port [port number]
```

Example Usage

Let's take the IP address 10.22.33.44 as an example.

Let's list the ports on this address.

```
nmap 10.22.33.44
```

The output we will get:

```
| PORT | STATE | SERVICE | | ---- | ----- | ----- | | 21 | OPEN | ftp | | 22 | CLOSE | ssh | | 80 | CLOSE | http |
```

STATE:

- OPEN: This port is defined and actively used.
- CLOSE: These ports are defined but closed for use.
- FORWARDED: The port is open, but it has been redirected to a different port instead of the default one. This means the service is still accessible, but through a different port number. For example, an FTP server normally runs on port 21, but if it appears as FORWARDED on port 37359, it means connections to 37359 are being forwarded to the actual FTP service running on port 21 inside the network.

SERVICE:

- Service running on this port in the background.

When we did a more detailed search:

```
nmap 10.22.33.44 -sV
```

The output we will get:

```
| PORT | STATE | SERVICE | VERSION | DESTINATION | | ---- | ----- | ----- | ----- | | 21 | OPEN | ftp | vsftpd 1.0.0 | 192.168.1.2 | | 22 | CLOSE | ssh | openssh 1.0.0 | 192.168.1.2 | | 80 | CLOSE | http | nginx 1.0.0 | 192.168.1.2 |
```

VERSION:

This section consists of 2 parts.

- Part 1: Name of the running application.
- Part 2: Version of this running application.

If we know the running service version and the port is open, we can infiltrate this system.

Command: node

This command allows you to run in-game scripts (with a .ts extension). It is not installed by default.

```
node myscript.ts
```

Command: nslookup

Finds the IP address of the target domain.

Usage

```
nslookup [domain]
```

Command: Nuclei

Nuclei is a fast, customizable vulnerability scanner.

```
nuclei -h <hosts>
```

Usage

To use Nuclei, you need a text document containing domain addresses.

Example: /domain_list.txt

```
example.com
domain.com
myanotherdomain.com
```

Then

```
nuclei -h /domain_list.txt
```

Command: openssl

Encrypts or decrypts text.

Usage for encryption

```
openssl -enc [text]
```

Usage for decryption

```
openssl -dec [text]
```

Command: ping

It checks whether it successfully sent a packet to the server and whether it received a response.

Usage

```
ping [ip address]
```

Command: pip

Pip is a package manager for Python. It is required for installing programs written in Python.

For install `apt-get install pip`

Installing python package `pip install <package name>`

Example `pip install sqlmap`

Command: python3

The **python3** command allows us to run a python script.

Usage

```
python3 [/destination/to/file]
```

Example

```
python3 /downloads/net_tree.py
```

You can find python scripts to run on hackdb.net

Command: rm

Deletes file/folder.

Usage

```
rm [file/folder name]
```

Command: ssh

- Secure Shell (SSH) is a protocol that provides a secure way to access a remote computer.
- It allows you to remotely log in to a system and execute commands as if you were sitting at the console.
- The important thing is flags.
- In ssh we don't use common usage.

- At `-h` flag you need to combine username and ip address (without port).
- Example: `root@192.168.1.1`

Usage

```
ssh -h [username@ip] -p [port]
```

Command: subfinder

`subfinder` is a subdomain discovery tool that returns valid subdomains for websites, using passive online sources. It has a simple, modular architecture and is optimized for speed. `subfinder` is built for doing one thing only – passive subdomain enumeration, and it does that very well.

```
subfinder -d <domain>
```

Examples

```
subfinder -d example.com
```

```
subfinder -d https://example.com
```

Command: sudo

It allows you to switch to the "root" user, which is the highest authorized user in computer.

If you want, switch directly to the root user or run a command with root authority.

Switch root user

```
sudo -su
```

Run command with root privileges

```
sudo [command]
```

Example:

```
sudo apt-get install metasploit
```

Command: weechat

Weechat allows you to chat online completely anonymously using just your username. It's often used for darknet.

Usage `weechat <ip address> <password>`

Command: whois

With the Whois command, you can find the person who owns a domain, IP address or email address.

Usage

```
whois [domain/ip address/email]
```

Python Module: Evil-Rm

Connect to a remote machine and execute a command.

Installing `pip install evil-rm`

Usage `evil-rm -i <ip address> -u <username> -H <hash>`

Important To use Evil-RM, you need to know the username and password hash of a user on the target device. For this, we recommend reviewing NTLM tools.

Example `evil-rm -i 47.189.132.4 -u alex -H 09bb63bf7635f9cfd50f022e7a3b0dba`

Python Module: Fern

A tool for cracking router passwords if the router model is publicly available.

Usage

- Download fern.py from hackdb.net.
- Install python `apt-get install python3`
- `python3 fern.py`
- Enter router model.

Example `python3 /home/user/downloads/fern.py Router Model > WN087A2`

Python Module: JWT Decoder

This exploit decodes JWT tokens and manipulates their payload to bypass security and escalate privileges. It's commonly used to exploit weak or absent JWT signature validation mechanisms.

Usage

- Download jwt_decoder.py from hackdb.net.
- Install python `apt-get install python3`
- `python3 jwt_decoder.py <jwt token>`

Example `python3 /home/user/downloads/jwt_decoder.py eyJ...`

Python Module: Kimai

Kimai is vulnerable to SameSite=Cookie=Vulnerability=session=hijacking. The attacker can trick the victim to update or upgrade the system, by using a very malicious exploit to steal his vulnerable cookie and get control of his session.

Kimai is known to be effective on firewalls.

Usage

- Download kimai.py from hackdb.net.
- Install python `apt-get install python3`
- Start listening on Wireshark.
- `python3 kimai.py <ip address>`
- Kimai may not be able to steal a cookie 100% of the time. If you can't get a cookie, try again.

Example `python3 /home/user/downloads/kimai.py 46.20.15.34`

Python Module: Net Tree

This exploit leverages an IP address as input to generate a complete network topology. Using active and passive scanning techniques, the tool identifies routers, switches, and connected devices, along with their open ports and services. The output is visualized using a tree-like structure for better readability.

Usage

- Download net_tree.py from hackdb.net.
- Install python `apt-get install python3`
- `python3 net_tree.py <ip address>`

Example `python3 /home/user/downloads/net_tree.py 46.20.15.34`

Python Module: Pret

PRET is a new tool for printer security testing developed in the scope of a Master's Thesis at Ruhr University Bochum. It connects to a device via network or USB and exploits the features of a given printer language.

Usage

- Download pret.py from hackdb.net.
- Install python `apt-get install python3`
- `python3 pret.py <ip address>`

Example `python3 /home/user/downloads/pret.py 46.20.15.34`

Important

- PRET requires the printer to be reachable via its LAN IP address with port 9100 (RAW printing) open. If port 9100 is closed, PRET cannot communicate with the printer.

Python Module: pyUserEnum

pyUserEnum is a tool for enumerate users on a subnet.

Usage

- Download pyUserEnum.py from hackdb.net.
- Install python `apt-get install python3`
- `python3 pyUserEnum.py <ip address>`

Example `python3 /home/user/downloads/pyUserEnum.py 46.20.15.34`

Python Module: Sqlmap

Sqlmap is a tool for testing and exploiting SQL injection vulnerabilities.

Installing `pip install sqlmap`

Usage `sqlmap -u <url> <options>` or `python3 sqlmap.py -u <url> <options>`

List of available options

- Listing tables: `sqlmap -u <url> -tables`
- Dump table data: `sqlmap -u <url> -dump -table <table name>`

Example usage
 Let's say our target is bestecommerce.com and there is a SQL vulnerability.

```
sqlmap -u bestecommerce.com -tables
```

Result | Table name | | ---- | | Products | | Users |

Then `sqlmap -u bestecommerce.com -dump -table Products`

Result | Product ID | Product Name | Price | | ---- | ---- | ---- | | 1 | Phone | 499\$ | 2 | Computer | 999\$

Python Module: WigleNet

WigleNet is a tool to find all devices in a specific location.

Usage

- Download wiglenet.py from hackdb.net.
- Install python `apt-get install python3`
- `python3 wiglenet.py -lat <latitude> -long <longitude>`

Example `python3 /home/user/downloads/wiglenet.py -lat 145.06 -long 68.592`

The Online World

Multiplayer is one big world that everyone shares. Every company site, home network, and person you see is a live target, and other operators (plus automated NPC crews) are working the same boxes at the same time as you.

A few things follow from that:

- **Loot runs out.** When a target gets robbed its loot is used up, and the box goes on a cooldown (usually a couple of hours) before it fills back up. If a box looks empty, come back later or move on to another one.
- **The world keeps moving while you are away.** NPC crews farm targets on their own, so the fat score you were saving can be gone by the time you reach it. Open the Network Monitor to see who is active on the wire.
- **Targets are not frozen.** A box you cracked last week will not look the same forever. Its open services can shift between raids, so scan it fresh every time instead of trusting the old way in.

The rest of this section walks the whole loop: find a target, break in, grab the loot, and turn it into clean money without getting caught.

Finding Targets

You can only attack something you can reach, and you reach a box by its **IP address**. Recon is simply the work of turning a name into an IP.

Three commands do most of the job:

- `nslookup <domain>` resolves a domain to an address. For a plain machine, that address is the box itself.
- `whois <domain>` shows who owns a domain, plus contact and mail hints that help later when you phish.
- `mxlookup <domain>` resolves the **mail** server. Mail almost never hides behind a CDN, so this is a dependable way to leak a real address.

Tip: `whois <IP>` looks up the person or machine using that address, while `whois <domain>` looks up the company that owns the name. If a name surprises you, that is why.

When a target hides behind a CDN

Sometimes `nslookup` says the host is behind Cloudfire and the origin is hidden. That means the address you got is only the CDN edge, and the edge cannot be exploited. You have two ways around it.

The mail record. Mail is usually not proxied:

```
mxlookup targetcorp.com
```

Passive DNS. A CDN can hide what a name resolves to today, but it cannot unpublish what the name already leaked. Every address the domain was seen on before it went behind the proxy stays on the record:

```
dnshistory targetcorp.com
```

Either one hands you the real **origin IP** to scan and exploit. If a zone was proxied from its very first record it has no history at all, which means that owner set it up properly and you need another angle.

Reputation opens doors

Higher-value targets stay out of reach until you have built a name. Reputation climbs as you pull off jobs, and the bigger the payout, the more reputation a target expects before it will even respond. Start small and work your way up.

Targets & Tiers

Not every target is equally hard. Boxes come in tiers, and the tier tells you how much work getting in will take.

- **Tier 1, direct.** Reachable straight from the internet. Scan the IP, find the open service, exploit it. Great for building reputation and starter cash. See *Tier 1: Direct Targets*.
- **Tier 2, VPN gated.** You cannot touch the real box from outside. First you get a VPN certificate (by phishing an employee or by cracking a captured hash), connect with `openvpn`, and only then can you exploit the target. See *Tier 2: VPN-Gated Targets*.
- **Tier 3, vaults.** Hidden targets that never appear until something leaks a lead. The prize sits deep inside a private network, so you breach the edge host and then **pivot** hop by hop toward the vault at the core. See *Tier 3: Vaults*.

Higher tiers pay far more, raise heat faster, and often need reputation before the target answers at all. Climb steadily; do not expect to walk into a vault on day one.

Each tier has its own walkthrough in this section, and *Operator Commands* lists every command the online world adds.

Tier 1: Direct Targets

A Tier 1 box answers the open internet. There is no gate: if you can resolve its address, you can attack it. This is where you learn the loop and build the reputation the higher tiers ask for.

The full run

1. **Pick a target.** Browse the directory or the Network Monitor, or follow a domain you turned up while digging.
2. **Resolve it.**

```
nslookup targetcorp.com
```

If the answer says the host sits behind Cloudfire, that address is only the CDN edge and cannot be exploited. Take the real origin off the mail record or the passive-DNS history instead:

```
mxlookup targetcorp.com
dnshistory targetcorp.com
```

3. Map the surface.

```
nmap 203.0.113.5
```

4. Sort the real doors from the traps.

```
probe 203.0.113.5
```

Only swing at a port marked **EXPOSED**. A **hardened** port is a dead end that costs you trace, and a **HONEYPOT** will get you caught. **probe** also tells you the attack style: a direct port exploit, or a reverse shell where you set LHOST and LPORT yourself.

5. **Exploit it.** Open **msfconsole**, select the module for that service, set the options **probe** pointed you at, and run it. The *Metasploit* section has the exact syntax.

6. **Take the loot.** Find the data package on the box and download it. The transfer moves the file rather than copying it, so once it lands on your machine nobody else can take that score.

7. Cover your tracks before you leave.

```
clearlogs targetcorp.com
```

The breach writes your public IP into the host's access log. Leaving it there feeds the defender's forensics and raises your heat.

8. Cash out.

```
appraise loot.dump
```

Then launder it or sell it. See *Appraising Loot* and *Cashing Out*.

What to expect

Tier 1 pays the least and is the quietest on your heat, which is exactly why it is the right place to grind reputation. The exploitable service is rolled per target and re-rolls when the box refreshes, so never assume last week's way in still works. Scan fresh every time.

Tier 2: VPN-Gated Targets

A Tier 2 company does not expose the box that holds the money. From outside you see the website and little else, and the exploit is refused until you are inside the corporate network. Getting inside means holding a **VPN certificate** for that specific company.

Step 1: Get a certificate

Two paths, and either one works.

Phish an employee. Recon the company for a named staff member: `whois` , the site's about and

contact pages, and `mxlookup` for the mail domain all help. Send them a booby-trapped document,

and when they open it you land a shell on their workstation. Their machine holds the corporate `.ovpn` config; download it.

Crack a captured hash. If you already pulled a password hash, from a `sqlmap` users dump on the public site or out of a phished inbox, forge it into a certificate:

```
crack targetcorp.com
```

The crack runs on a real clock scaled by tier and the server enforces the wait, so start it and go do something else. It drops a cert file when it finishes.

Step 2: Bring the tunnel up

```
openvpn corp-vpn.ovpn targetcorp.com
```

The tunnel belongs to **the terminal that opened it**. That terminal is now busy holding the line, so open a second one and work from there. Close the first and the tunnel drops.

Step 3: Work the box from inside

Now the target answers. Run the Tier 1 routine, but from inside the tunnel:

```
nmap 203.0.113.5
probe 203.0.113.5
```

Exploit the exposed service, take the package, and run `clearlogs` before you let the tunnel go.

Things that bite people

- A cert is bound to **one instance** of the target. Once the box is looted and rotates, the old cert is dead and you need a fresh one.
- A stolen cert opens **that company only**. It is not a skeleton key for other targets.
- A **generic cert** bought on the GreyMarket works on any matching target but is used up on the first connection.
- Tier 2 raises heat noticeably faster than Tier 1. Rent proxy hops before you make a habit of it.

Tier 3: Vaults

Tier 3 is the top of the ladder: a private vault behind a corporate edge, holding encrypted loot worth more than everything below it put together. Nothing about it is automatic. You need reputation, two different keys, a pivot through the internal network, and a decoder on the way out.

Step 0: Find it

Vaults are **not public**. They never show up in the directory and you cannot stumble onto one by scanning. You surface a vault by working the tier below it:

- **Loot leaks a lead.** Some targets carry a pointer to a hidden one. Rob them and the vault appears in your directory and network view.
- **Intel leaks a lead.** Pivot onto an internal NPC box and download the intel file sitting on it. That surfaces the lowest-tier hidden target you have not found yet, so the ladder unlocks roughly in order.

Step 1: Have the reputation

A vault checks your name before it answers at all. If it tells you the target is out of your league, the number in the refusal is what you still have to earn on smaller jobs.

Step 2: Both keys, both appraised

A vault wants two things at once:

- a **vault VPN certificate**, and
- a **vault access token**.

Both drop from raids or trade in the GreyMarket black market. A key that comes out of a raid arrives encrypted, and an encrypted key gets rejected, so appraise each one before you rely on it:

```
appraise vault-token.key
```

Step 3: Tunnel in, then arm the auxiliary

```
openvpn vault-vpn.ovpn vaultcorp.com
```

From a second terminal, hand the token to the target's metasploit access auxiliary:

```
authkey vault-token.key vaultcorp.com
```

The exploit stays refused until that auxiliary is armed. The token is single-use, but it is only spent once a foothold actually lands, so getting caught by a trace does not waste it.

Step 4: Breach the edge, then pivot

Exploit the edge host the usual way, then start working inward:

```
pivot vaultcorp.com
```

That lists the **frontier**: the hosts reachable from where you are standing. **Only one of them is real.** The others are dead ends that cost you trace for touching them. Hosts you have already captured leak config and log lines, and reading those is the only honest hint about which candidate opens the next layer.

Hop onto the one you picked:

```
pivot vaultcorp.com dbnode01
```

A correct hop opens the next layer and drops loot: cheap decoy packages on the way in, the real vault at the core. After every single hop:

```
clearlogs vaultcorp.com
```

Step 5: Get out and unscramble it

Vault loot exfiltrates **encrypted** and is worth nothing until it is decoded. You need a decoder matching the vault's scheme, from a raid drop or the black market:

```
decode vault.dump vault-aes.dec
```

Then appraise it and fence it like anything else.

Things that bite people

- Skipping **clearlogs** between hops stacks trace fast, and a vault run is long enough for it to add up.
- Poking frontier hosts at random is the most common way to get traced on a vault. Read the leaked config first.
- Line up the decoder **before** the run if you can. Sitting on undecodable loot while your heat is high is expensive.

Breaking In

Once you have a reachable IP, the break-in is a short routine.

1. **Scan the box.** See what is listening:

```
nmap 203.0.113.5
```

2. **Probe before you swing.** Not every open port is a real door. **probe** sorts them for you:

```
probe 203.0.113.5
```

- **exposed** ports are genuine, exploitable services.
- **decoys** look open but lead nowhere. Poking one wastes time and adds a little trace.
- **honeypots** are traps. Hitting one spikes your trace hard and can get you caught, so leave them alone.

3. **Exploit the real service.** Open Metasploit, aim it at the exposed port, and run the matching exploit to land a shell. The *Metasploit* section covers the exact commands.

The exploitable service is randomized per target, so let **probe** tell you which port is real rather than assuming it is always the same one.

VPN Certificates

Tier 2 boxes do not answer the open internet. You have to be inside the company network first, and that means holding a valid **VPN certificate** for that specific target. This page covers the certificates themselves; *Tier 2: VPN-Gated Targets* walks the whole job start to finish.

There are two ways to get one.

Phish an employee. Find a staff member (recon and the company site help here), send them a booby-trapped mail, and when they bite you land on their workstation. From there you can download the company **VPN config file** to your machine.

Crack a captured hash. If you pulled a password hash (from a `sqlmap` users dump or a phished inbox), turn it into a certificate:

```
crack targetcorp.com
```

Cracking takes real time based on the tier, so start it and let it run.

With a cert in hand, bring the tunnel up:

```
openvpn corp-vpn.ovpn targetcorp.com
```

A few things worth knowing about the tunnel:

- It is **tied to the terminal** that opened it. Close that terminal and the tunnel drops, so keep this one holding the line and work from another terminal.
- A cert is bound to **one instance of the target**. It keeps working for as long as that box stays put, but the moment the target is looted and rotates, the old cert stops working and you need a fresh one.
- A stolen cert only opens the company you took it from. It is not a skeleton key for other targets.

Keys & Decoders

Beyond VPN certs, two kinds of tools open doors and unlock loot.

Access keys get you into gated targets, and they come in two flavors:

- **VPN certificates** ride a tunnel up. Use them with `openvpn` :

```
openvpn <file> <target>
```

- **Access tokens** authenticate you straight to a service. Use them with `authkey` :

```
authkey <file> <target>
```

Every key is stamped with the tier it unlocks, so a Tier 2 key opens Tier 2 doors.

Decoders unscramble encrypted loot. Each decoder matches one scheme, and you feed it to `decode` (see *Encrypted Loot*).

Where tools come from

- **Raids.** Keys and decoders sometimes drop as bonus loot alongside a heist.
- **The GreyMarket black market.** Once unlocked you can buy the exact key or decoder you need and sell the spares you pull. See *The GreyMarket*.

One note on bought certificates: a **generic cert** is a clean, ready-made certificate from the market rather than one you stole. It works on any matching target and arrives already identified, but it is used up on the first connection, so treat it as a convenience rather than a permanent key.

Vaults & Pivoting

Tier 3 targets keep the real prize on an internal network you cannot see from outside. You breach the edge host like any other box, then work inward one layer at a time. That is **pivoting**.

List the hosts you can currently reach from your foothold:

```
pivot targetcorp.com
```

Each layer shows a short **frontier** of candidate hosts, but **only one of them is actually exploitable**. The rest are dead ends that cost you trace if you poke them. Captured hosts leak config and log lines, so read those hints to work out which candidate is the real one.

Hop onto the host you picked:

```
pivot targetcorp.com dbnode01
```

The right host opens the next layer and drops loot: cheap decoy scores on the way in, and the encrypted vault at the core. After every hop, cover your tracks:

```
clearlogs
```

Keep going until you reach the core, then loot the vault and get out.

Loot Packages

What you steal comes out as an **opaque data package**: a raw dump with a realistic name like

```
backup.sql.gz
```

,

```
users_dump.sql
```

, or

```
wallet.keystore
```

. There is no viewer for it and you cannot

open it. Double-clicking just tells you it is an encrypted package.

That is on purpose. A raw dump does not advertise its worth. Two things can be true of a package:

- **Un-appraised.** You do not know what it is worth yet. Appraise it (see *Appraising Loot*).
- **Encrypted.** It is scrambled and has to be decoded first (see *Encrypted Loot*).

Moving a package to your machine is a **transfer, not a copy**. Once you pull it, it is gone from the box, so nobody else can grab that same loot.

Appraising Loot

A fresh package is a mystery until you look inside. From a terminal:

```
appraise <file>
```

Appraising reveals the package **type and real value**, and rewrites the file so you can read what you actually pulled (a preview of the records, config, keys, and so on). Only then does it carry a price you can act on.

Un-appraised loot cannot be sold or laundered for its true value, so appraise first, then cash out.

Encrypted Loot

Some packages come out **encrypted**, because hardened targets ship their crown jewels scrambled. An encrypted package cannot be appraised for full value or fenced until you decrypt it.

You need a matching **decoder** for the package scheme. Decoders drop as bonus loot from raids and trade on the GreyMarket black market. With one in hand:

```
decode <file> <decoder>
```

Once it is decoded, appraise it and fence it like any other package.

Cashing Out

Loot is not money until you move it. Raw packages are **hot** (traceable), so you launder or sell them instead of depositing them directly.

- **Bank and financial loot** (checks, wire logs) gets **laundered**. A cut is taken, and the cut grows while your heat is high.
- **Data loot** (database dumps, credentials, wallets) gets **sold** on the GreyMarket. List it yourself, or let an NPC broker snap up fairly-priced listings that have sat for a while.

The two pay in different money. Laundering puts dollars in your bank; a GreyMarket sale pays **฿** into your market balance, which buys tools on the market but converts back to dollars at a terrible rate. Always appraise before you list, because an un-appraised package fences for nothing.

The GreyMarket

greymarket.net is where operators trade. It runs on its own currency, **฿**, and has three rooms.

฿ and your bank are not the same money

Everything on GreyMarket is priced in **฿**. Selling pays **฿** into your market balance, and every purchase here spends it. Your LCB bank account is never touched, and there is no way to put dollars in.

The balance pill in the top right opens **cash out**, the only way to move **฿** into your bank. The street rate is savage: **฿**1,000 lands about \$6. That is the point. **฿** is worth its face value as long as it stays on the market, and almost nothing once it leaves, so the smart play is to reinvest it in tools rather than cash it out.

The flea market

The open floor is player to player. Hit **Sell an item**, pick a loot file from your own machine, and set a price. The dialog shows recent low, average, and high prices for that kind of item so you can price it sanely. Other operators buy your listings, and you can buy theirs. Selling hands the file to the buyer and removes it from your machine. If nobody bites, an NPC broker may eventually clear a fairly-priced listing so the floor keeps moving.

Listing slots. You start with room for **3** live listings. Slots are earned by trading: every time your lifetime **฿** turnover crosses the next threshold, the market trusts you with one more. The counter under the toolbar shows how far off the next one is.

The exploit trader

A rotating shelf of basic **.exp** files, restocked on a timer. Mostly common, some uncommon, the occasional single-stock rare. It never carries epic or legendary, because those only ever drop from Tier 3 loot.

The black market

A locked back room for **access keys and decoders**. Getting in costs a one-time **฿16,667** out of your market balance, so it is a gate you can only reach by actually trading. Once it is unlocked you can:

- **Buy tools** from a catalog: the exact key or decoder you need for a target, priced by tier.
- **Sell tools** you looted or no longer need, fenced back for ฿ at a flat haircut.

Bought tools arrive already identified and land in your downloads, ready to use.

Heat & Hot Loot

Every heist raises your **heat**. Heat is the world remembering how loud you have been.

- Freshly stolen loot is **hot**. Handling it while your heat is high costs you more: a bigger laundering cut and worse fences.
- Heat cools on its own over time. If a job feels expensive to cash out, lie low and let it fall.
- Bigger scores and hardened targets raise heat faster but pay far more. That is the trade you are making.

Stack too much heat and the wire turns dangerous: other operators watching a box can catch you on the way out. Balance greed against exposure.

Getting Traced

Every noisy move raises a **trace** meter on the job you are running: poking decoys, tripping a honeypot, or lingering too long on a box. Let the meter fill and you get **traced**.

When that happens the run is burned on the spot: the connection drops, that target locks you out for a while, and your heat goes up.

How to stay clean:

- Trust **probe** and do not swing at decoys or honeypots.
- Run **clearlogs** after you work a host, especially while pivoting.
- Rent proxy hops to soak up heat and trace (see *Proxies & Rigs*).
- If a job is getting hot, back out and let things settle instead of pushing your luck.

Proxies & Rigs

Two upgrades quietly make every job safer and faster. Both are sold under **Upgrades** in the Black Market back room on **greymarket.net**, and both are paid for in ฿ you earned on the trading floor.

Proxy hops. Each rented hop routes your traffic through another relay, which lowers the heat you pick up per job. More hops means a quieter footprint, and it is your main defense if you plan to hit big targets often.

Rig tier. Upgrading your rig makes hacks and downloads run faster. Higher tiers shorten the slow parts of a raid, like cracking and exfiltrating, which matters when loot is contested and seconds count.

Spend early profits here. A calmer, faster operator loses less loot to rivals and gets caught far less often.

Your Own Infrastructure

Everything you break into, you can also run yourself. Rent a box, rent a domain, serve a site from it, and you become a target like any company in this world. This page is the defending side of the manual.

Renting a box and a name

Hosting providers sell servers from their own dashboard, and registrars sell domains by the day or by the month. Both are prepaid for a fixed window: a lapsed rental stops serving, and a lapsed domain eventually goes back on the market, so renew from the dashboard before it does. You can pay out of your clean LCB balance or with a card, and a card you stole spends exactly as well as your own.

Where you buy the name matters, because `whois` reports it:

- **Namecheep** is the cheapest and publishes your registrant record, so anyone who looks the domain up sees the contact details behind it.
- **GoDiddy** costs more per day and includes WHOIS privacy on every domain it sells, so the same lookup comes back redacted.

Privacy hides who you are. It does nothing about where your box is, which is the next section.

Pointing the name at the box

The registrar and Cloudfire both give you a DNS manager for the names you own: A, CNAME, MX and TXT records. Point the apex (`@`) A record at your server's address and the site becomes browsable at that domain. Delete that record and the site goes dark.

Keeping your origin hidden

Sitting a name behind Cloudfire replaces the public answer with a CDN edge address. `nslookup`

hands out the edge, `whois` reports no address at all, and the edge itself cannot be exploited. The protection is real, but it is only ever as good as your zone's history. Three rules:

1. **Proxy the record from the very first save.** Cloudfire's add-a-site form does this for you. In a registrar's DNS panel the Proxied box starts unchecked, and saving your address that way, even once, even for a minute, is the most common way an origin gets leaked. The world records every address a name is seen on while it is unproxied, and `dnshistory yoursite.com` reads that record back forever.

Switching the proxy on afterwards hides today's answer and unpublishes nothing. A zone proxied from

its first record has no history at all, and that is the state you want. If you have already leaked an address, the only real fix is to move to a box with a new one and never publish it in the clear.

2. Every name in the zone counts. A `dev` or `staging` record left unproxied gives away the same machine the apex would, and `subfinder` lists every name in your zone, so a forgotten one will be found.

3. Do not publish an MX that points at your own box. Mail records cannot be proxied, and `mxlookup` reads the mail host's address straight out of your zone. That is how company origins get taken, and yours is no different.

Hardening what sits behind it

Cloudflare's Security tab sells defense layers per server, bought once each with clean funds:

Layer	What it does
Intrusion Detection (IDS)	Logs every attempt and alerts you. Does not block on its own.
Network Firewall	Filters a service port off the open internet, sealing one path in.
Web Application Firewall	Inspects web traffic and blocks exploits aimed at the web tier.
Honeypot Trap	Plants a decoy service. Whoever attacks it is traced on the spot.
Managed Auto-Patch	Keeps every service patched, so there is no exploitable weakness left.
Smart Firewall (IPS)	One wrong move and the attacker's address is banned from the box for good.

The same panel lists recent intrusions with the address each attempt came from, which is your early warning that somebody has found you.

Layers are expensive on purpose, so buy for the box you have: a machine serving a site wants the web tier covered, a machine holding anything valuable wants Auto-Patch. And keep the order of operations straight. Hide the address first. A hardened box whose origin is sitting in passive DNS is only a slower target, not a safe one.

Working From a Rented Server

Every loud thing you do puts an address on the wire. It lands in the target's access log, in the surveillance feed of anyone watching that host, and in any per-address ban the box hands out. By default that address is your own machine's.

A rented box changes it. Open a shell on a server you rent and, for as long as that shell is open, everything you launch leaves through it:

```
ssh -h operator@203.0.113.5
```

From that moment the target sees the rented box's address instead of yours. Close the last shell and you are back on your own connection. Your own machine still reports its real address in `ifconfig` : only the work you launch from the server exits through the server.

Why it is worth the rent

- **A ban hits the box, not you.** A Smart Firewall bans the address that made the wrong move, permanently. If that was a rental you burn the rental and take another one. If it was your own line,

that target is closed to you for good.

- **Surveillance names the box.** Operators watching a host see the source address of every attempt, so working through a rental keeps yours out of their capture.
- **A line you forget to wipe points somewhere else.** `clearlogs` is still the habit to keep, but anything you miss names the rented box.

Practical detail

- Hold as many shells as you like. You keep leaving through a box until the last one closes.
- The routing lives with your session: a disconnect drops it, and it comes back with your shell.
- A lapsed rental routes nothing. Renew before you lean on it.
- Joining a foreign Wi-Fi network moves your address the same way. It is the cheap answer to a ban when you have no box to work from.

Keep the box that serves your site separate from the box you work from. A site box has a domain pointing straight at it, which makes it the easiest address of yours for a rival to find, and that is the last address you want sitting in a victim's log.

Operator Commands

These commands exist only in the online world. Running one in singleplayer just tells you it is multiplayer only.

Recon and surveillance

Command	What it does	----	----	<code>probe <ip></code>	Deep vuln scan. Classifies every port as
	exposed, hardened, or honeypot. Slower and louder than	<code>nmap</code> .		<code>dnshistory <domain></code>	Passive
	DNS. Lists every address the domain was seen on, which is how you get behind a CDN.				
<code>watch <target></code>	Start surveilling a target.	<code>watch off <target></code>	Stop surveilling it.		
<code>watch who <target></code>	Sweep the target for other operators watching it.				

`watch` puts a target on your wire: breach attempts and loot exfiles on that host stream into the

Network Monitor app, where you can start a capture to record them. Nothing is broadcast globally, so only operators who bothered to watch see the traffic. Run `watch who` on a box before you hit it to find out whether somebody is watching you back.

Access

| Command | What it does | | ---- | ---- | | **crack <target>** | Turn a captured password hash into a VPN certificate. Runs on a real clock. | | **openvpn <cert> <target>** | Bring up the tunnel. Held by the terminal that opened it. | | **authkey <token> <target>** | Arm a vault's access auxiliary with an access token, before you run the exploit. |

Inside the network

| Command | What it does | | ---- | ---- | | **pivot <target>** | List the frontier hosts you can reach from your foothold. | | **pivot <target> <host>** | Hop onto a host and open the next layer. | | **clearlogs <target>** | Wipe your IP out of that host's access log. Run it after every hop. |

Loot and money

| Command | What it does | | ---- | ---- | | **appraise <file>** | Identify a package: reveals loot's real value, and unlocks a key or decoder for use. | | **decode <loot> <decoder>** | Decrypt encrypted loot with a matching decoder. Single-use. | | **launder <check>** | Run a stolen check through the Mixer into clean funds. The cut grows with your heat. | | **market** | List the black-market catalog. | | **market buy <id>** | Buy a listing with clean bank funds. | | **market sell <file>** | Sell a key or decoder you are holding. |

Only bank checks go through the Mixer. Data loot (dumps, credentials, wallets) is **sold** on **greymarket.net** instead. Either way, appraise it first: an unidentified package fences for nothing.

Leaving your mark

| Command | What it does | | ---- | ---- | | **deface <message>** | Overwrite the homepage of the host you have a shell on. |

deface only works from inside a shell on a target that actually serves a site. The write is persistent and global: every player who visits that homepage sees your message until the host resets on its own. It pays nothing, so it is pure reputation among operators.